

Reviewer Pack — Automorphism Group Generator Count Bounded by ABP Width for ...

Entry e9123a8ea921 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 11:43:29 UTC

Automorphism Group Generator Count Bounded by ABP Width for Symmetric CNFs

Entry ID: e9123a8ea921 Verdict: INCONCLUSIVE Recorded: 2026-05-10 11:43:29 UTC

1. Conjecture

Field A (mathematical branch): Group Theory (Permutation Group Generators) **Field B** (complexity object): Algebraic Branching Programs (ABP Width)

Statement:

For any symmetric CNF formula Φ with automorphism group G , the minimal width of an ABP representing Φ is exactly the minimal number of generators required to generate G .

Rationale (proposer’s reasoning):

Symmetric CNFs have automorphism groups that capture their symmetry. The minimal generating set size of G reflects the structural complexity of the symmetry, which could influence the minimal AB. width needed to represent the formula, as ABPs are tied to permutation-based computations.

Taxonomy category: BARRINGTON_ALG (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e9df3f581bf437c5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def matrix_mult(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def matrix_power(M, p):
    result = [[1 if i == j else 0 for j in range(len(M))] for i in range(len(M))]
    base = M
    while p > 0:
        if p % 2 == 1:
            result = matrix_mult(result, base)
        base = matrix_mult(base, base)
        p //= 2
    return result

def is_permutation(p):
    n = len(p)
    visited = [False] * n
    for i in range(n):
        if not visited[p[i]]:
            visited[p[i]] = True
        else:
            return False
    return True
```

```

def find_cycle_length(p, start):
    cycle = []
    current = start
    while True:
        cycle.append(current)
        current = p[current]
        if current == start:
            break
    return len(cycle)

def automorphism_group(G):
    n = len(G)
    group = [list(range(n))]
    for i in range(n):
        for j in range(i + 1, n):
            if G[i][j] != G[j][i]:
                return None
    for p in itertools.permutations(range(n)):
        if is_permutation(p):
            valid = True
            for i in range(n):
                if not all(G[p[i]][p[j]] == G[i][j] for j in range(n)):
                    valid = False
                    break
            if valid:
                group.append(list(p))
    return group

def abp_width(G):
    n = len(G)
    generators = []
    for i in range(n):
        for j in range(i + 1, n):
            if G[i][j] != G[j][i]:
                return None
    for p in itertools.permutations(range(n)):
        if is_permutation(p):
            valid = True
            for i in range(n):
                if not all(G[p[i]][p[j]] == G[i][j] for j in range(n)):
                    valid = False
                    break
            if valid:
                generators.append(list(p))
    return len(generators)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    variables = list(range(n))
    clauses = []
    for _ in range(2 * n):
        a, b = random.sample(variables, 2)
        clause = [a, -b]
        if random.choice([True, False]):
            clause[1] *= -1

```

```

        clauses.append(clause)

G = [[0] * n for _ in range(n)]
for i in range(n):
    for j in range(i + 1, n):
        count = sum(1 for clause in clauses if (i in clause and not j in clause) or (j in clause and not i in clause))
        G[i][j] = count
        G[j][i] = count

group = automorphism_group(G)
if group is None:
    return {
        "metric_name": "ABP Width",
        "metric_value": None,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

k = len(group) - 1
w = abp_width(G)
if w is None:
    return {
        "metric_name": "ABP Width",
        "metric_value": None,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

return {
    "metric_name": "ABP Width",
    "metric_value": w == k,
    "instances_tested": 1,
    "conjecture_holds": w == k,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        support_fraction = 1.0
    else:
        support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={sum(r['metric_value'] for r in results) / len(results)}")
    else:

```

```

first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing evaluation of support fraction or counterexamples. | next: Increase timeout duration and re-run test with extended computation limits

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	108100
2	preregistration	ollama_remote	qwen3:8b	0	0	26084
3	novelty	ollama_remote	qwen3:8b	0	0	21017
4	novelty	ollama_remote	qwen3:8b	0	0	15968
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16170
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14115
7	judge	ollama_remote	qwen3:8b	0	0	15839

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 217292 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/e9123a8ea921.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e9123a8ea921.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/e9123a8ea921.tar.gz (if generated)